

```
data class ChatAskRequest(  
    val sessionId: String? = null,  
  
    @field:NotBlank(message = "Must not be blank")  
    @field:Size(max = 4000, message = "Must not exceed 4000 characters")  
    val question: String = "",  
  
    val namUser: String? = null,  
  
    val context: Map<String, String>? = null,  
)
```

E o controller fica essencialmente:

```
</> Kotlin   
  
@RestController  
@RequestMapping("/api/v1/chat")  
class ChatController(  
    private val chatService: ChatService,  
) {  
  
    @PostMapping(  
        value = ["/ask"],  
        produces = [MediaType.TEXT_EVENT_STREAM_VALUE],  
    )  
    fun ask(  
        @Valid @RequestBody request: ChatAskRequest,  
        @RequestHeader("pag-app-name", required = false) appName: String?,  
        @RequestHeader("x-risk-chat-token", required = false) riskChatToken:  
        @RequestHeader("x-correlation-id", required = false) correlationId  
    ): Flow<ChatStreamEvent> {  
        return chatService.processQuestion(  
            request = request,  
            appName = appName,  
            correlationId = correlationId,  
        )  
    }  
}
```

A parte realmente interessante é o `ChatService`.

1. Primeiro vamos definir o evento SSE

Para não complicar, eu usaria:

</> Kotlin



```
data class ChatStreamEvent(  
    val type: String,  
    val sessionId: String,  
    val content: String? = null,  
)
```

Então poderemos emitir:

```
type = "session"  
type = "chunk"  
type = "done"
```



Por exemplo:

</> JSON



```
{  
  "type": "chunk",  
  "sessionId": "abc",  
  "content": "Com base"  
}
```

e depois:

</> JSON



```
{  
  "type": "chunk",  
  "sessionId": "abc",  
  "content": " na análise"  
}
```

2. processQuestion usando channelFlow

Aqui está uma primeira implementação que eu usaria para você entender o comportamento:

</> Kotlin



```
@Service
class ChatService(
    private val sessionMemoryRepository: SessionMemoryRepository,
    private val sessionIndexRepository: SessionIndexRepository,
) {

    fun processQuestion(
        request: ChatAskRequest,
        appName: String?,
        correlationId: String?,
    ): Flow<ChatStreamEvent> = channelFlow {

        val sessionId =
            request.sessionId ?: UUID.randomUUID().toString()

        val chatHistoryId =
            UUID.randomUUID().toString()

        val answerBuilder = StringBuilder()

        // 1. Primeiro evento informa a sessão
        send(
            ChatStreamEvent(
                type = "session",
                sessionId = sessionId,
            )
        )

        try {

            // 2. Busca contexto anterior da conversa
            val previousContext =
                sessionMemoryRepository.getContext(sessionId)

            // 3. Gera uma resposta fake
            val completeAnswer =
                generateMockResponse(
                    question = request.question,
                    context = request.context,
                    previousContext = previousContext,
                )

            // 4. Simula o streaming do agente
            completeAnswer
                .chunked(20)
                .forEach { chunk ->

                    delay(200)

                    answerBuilder.append(chunk)

                    // ...
                }
        }
    }
}
```

```
        send(
            ChatStreamEvent(
                type = "chunk",
                sessionId = sessionId,
                content = chunk,
            )
        )
    }

    // 5. Agente terminou de responder

    val turn = ChatTurn(
        chatHistoryId = chatHistoryId,
        sessionId = sessionId,
        namUser = request.namUser,
        namSystem = appName,
        question = request.question,
        answer = answerBuilder.toString(),
        createdAt = Instant.now(),
        persistStatus = PersistStatus.PENDING,
    )

    // 6. Persiste memória operacional
    sessionMemoryRepository.append(turn)

    // 7. Atualiza índice de sessões do usuário
    request.namUser?.let { user ->
        sessionIndexRepository.touch(
            userId = user,
            sessionId = sessionId,
        )
    }

    // 8. Informa ao cliente que terminou
    send(
        ChatStreamEvent(
            type = "done",
            sessionId = sessionId,
        )
    )

} catch (ex: CancellationException) {

    /*
     * Cliente fechou o SSE.
     *
     * Nesta implementação simplificada:
     * NÃO salvamos um turno incompleto.
     */

    throw ex
}
```

```
}  
}
```

Esse código é pequeno, mas já representa praticamente todo o fluxo que estamos discutindo.

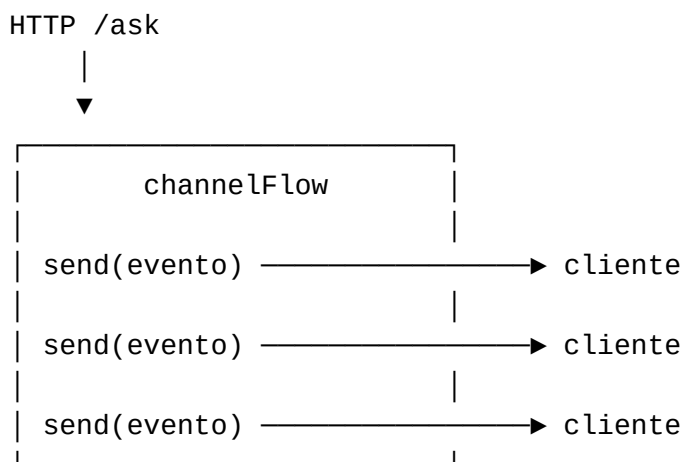
3. Visualize o `channelFlow`

Essa é a parte mais importante.

Quando entra:

```
</> Kotlin  
  
channelFlow {
```

você pode imaginar que foi aberto um canal:



Então:

```
</> Kotlin  
  
send(  
    ChatStreamEvent(  
        type = "session",  
        sessionId = sessionId  
    )  
)
```

já manda informação para o cliente.

A função **não terminou**.

Depois:

```
</> Kotlin  
  
send(chunk1)
```



cliente recebe.

Depois:

```
</> Kotlin  
  
send(chunk2)
```



cliente recebe.

Depois:

```
</> Kotlin  
  
send(chunk3)
```



cliente recebe.

E somente quando o bloco:

```
</> Kotlin  
  
channelFlow {  
    ...  
}
```



terminar, o Flow termina e a conexão SSE é encerrada.

4. Exemplo real de execução

Usuário envia:

```
</> JSON
```



```
{  
  "question": "Mostre o dossiê do cliente",  
  "namUser": "bruno",  
  "context": {  
    "customerId": "123456"  
  }  
}
```

Sem:

```
sessionId
```



Então:

```
</> Kotlin  
  
val sessionId =  
    request.sessionId ?: UUID.randomUUID().toString()
```



gera:

```
sessionId = S1
```



Depois:

```
</> Kotlin  
  
val chatHistoryId =  
    UUID.randomUUID().toString()
```



gera:

```
chatHistoryId = H1
```



Temos:

```
Usuário: Bruno
```



```
Session:  
S1
```

```
Turn:  
H1
```

5. Primeiro send

Executamos:

```
</> Kotlin   
  
send(  
    ChatStreamEvent(  
        type = "session",  
        sessionId = sessionId  
    )  
)
```

O frontend recebe:

```
</> JSON   
  
{  
  "type": "session",  
  "sessionId": "S1"  
}
```

Isso é útil principalmente quando é uma conversa nova.

O frontend agora sabe:

esta conversa virou a session S1 .

Na próxima pergunta ele pode enviar:

```
</> JSON   
  
{  
  "sessionId": "S1",  
  "question": "E quais foram os alertas?"  
}
```

6. Melhorando o generateMockResponse

Na imagem você tem aproximadamente:

```
</> Kotlin 
```



```
private fun generateMockResponse(question: String): String {  
  
    val q = question.lowercase()  
  
    return when {  
  
        q.contains("dossiê") || q.contains("dossie") ->  
            ...  
  
        q.contains("alerta") ->  
            ...  
  
        q.contains("transação") || q.contains("pix") ->  
            ...  
  
        else ->  
            ...  
    }  
}
```

Podemos melhorar isso para usar também o `context` da request:

⌕ Kotlin



```

private fun generateMockResponse(
    question: String,
    context: Map<String, String>?,
    previousContext: String?,
): String {

    val normalizedQuestion =
        question
            .trim()
            .lowercase()

    val customerId =
        context?.get("customerId") ?: "não informado"

    return when {

        normalizedQuestion.contains("dossiê") ||
            normalizedQuestion.contains("dossie") -> {

            """
            Com base na análise do dossiê:

            Cliente: $customerId

            1. **Status**: Em análise
            2. **Alertas**: 3 alertas de movimentação atípica
            3. **Valor total**: R$ 523.450,00
            4. **Risco**: Elevado

            Foram encontradas movimentações que precisam de análise adicional
            """.trimIndent()
        }

        normalizedQuestion.contains("alerta") -> {

            """
            Foram identificados 3 alertas para o cliente $customerId:

            - Aumento de 340% no volume transacional
            - Fracionamento recorrente de valores
            - Transações realizadas em horários atípicos

            Recomenda-se análise detalhada das movimentações.
            """.trimIndent()
        }

        normalizedQuestion.contains("transação") ||
            normalizedQuestion.contains("transacao") ||
            normalizedQuestion.contains("pix") -> {

            """
            Transações identificadas para o cliente $customerId:
            """
        }
    }
}

```

```

        transacoes identificadas para o cliente $customerId:

        - **Total**: 47 transações
        - **Valor movimentado**: R$ 892.150,00
        - **Maior transação**: R$ 154.357,00
        - **Tipo da maior transação**: PIX
        """.trimIndent()
    }

    normalizedQuestion.contains("contexto") -> {

        """
        Contexto recebido nesta requisição:

        ${context ?: "Nenhum contexto adicional informado."}

        Contexto anterior da sessão:

        ${previousContext ?: "Esta sessão ainda não possui histórico."}
        """.trimIndent()
    }

    else -> {

        """
        Consulta processada.

        Pergunta recebida:
        "$question"

        Para testar melhor o fluxo, tente perguntas relacionadas a:
        - dossiê
        - alertas
        - transações
        - PIX
        """.trimIndent()
    }
}
}
}

```

Agora o mock já começa a parecer um agente de verdade.

7. Mas o mais interessante é como fazemos streaming

O método retorna a resposta inteira:

</> Kotlin



```
val completeAnswer = generateMockResponse(...)
```

Por exemplo:

Com base na análise do dossiê:

Cliente: 123456

1. Status: Em análise
2. Alertas: ...

Então fazemos:

</> Kotlin

```
completeAnswer  
    .chunked(20)
```

Isso transforma:

"Com base na análise do dossiê..."

em algo aproximadamente assim:

```
[  
    "Com base na análise ",  
    "do dossiê:\n\nCliente",  
    ": 123456\n\n1. **Stat",  
    "us**: Em análise..."  
]
```

Depois:

</> Kotlin

```
forEach { chunk ->  
  
    delay(200)  
  
    answerBuilder.append(chunk)  
  
    send(...)  
}
```

Simulamos exatamente o comportamento do agente.

8. Durante a execução

Imagine:

```
completeAnswer:
```



```
"Com base na análise do dossiê..."
```

O `StringBuilder` começa:

```
""
```



Primeiro chunk:

```
"Com base na análise "
```



Então:

```
answerBuilder:
```



```
"Com base na análise "
```

e:

```
SSE —————> "Com base na análise "
```



200 ms depois:

```
chunk:
```



```
"do dossiê..."
```

Agora:

```
answerBuilder:
```



```
"Com base na análise do dossiê..."
```

e:

```
SSE —————> "do dossiê..."
```



Esse processo continua até acabar.

9. Por que existe o answerBuilder ?

Porque existe uma diferença entre:

o que enviamos ao frontend



e:

o que queremos armazenar no Redis



O frontend recebe:

```
"Com base "  
"na análise "  
"do dossiê "  
"..."
```



Mas o Redis precisa receber:

```
</> JSON  
  
{  
  "question": "Mostre o dossiê",  
  "answer": "Com base na análise do dossiê..."  
}
```



Então:

```
</> Kotlin  
  
answerBuilder.append(chunk)
```



reconstrói a resposta final.

10. E então criamos o turno

Depois que o loop termina:

```
</> Kotlin
```



```
val turn = ChatTurn(  
    chatHistoryId = chatHistoryId,  
    sessionId = sessionId,  
    namUser = request.namUser,  
    namSystem = appName,  
    question = request.question,  
    answer = answerBuilder.toString(),  
    createdAt = Instant.now(),  
    persistStatus = PersistStatus.PENDING,  
)
```

Por exemplo:

```
ChatTurn  
|  
├─ chatHistoryId = H1  
├─ sessionId = S1  
├─ namUser = Bruno  
├─ namSystem = fraud-analysis  
├─ question = "Mostre o dossiê"  
├─ answer = "Com base na análise..."  
├─ createdAt = ...  
└─ persistStatus = PENDING
```

Agora finalmente entramos no Redis.

11. Eu esconderia as cinco operações Redis atrás de append

Em vez do `chatService` saber que existem:

```
iara:memory  
iara:order  
iara:pending  
iara:context
```

eu faria:

```
</> Kotlin  
  
sessionMemoryRepository.append(turn)
```

e deixaria o repository cuidar disso.

Algo conceitualmente assim:

⌕ Kotlin




```
@Repository
class SessionMemoryRepository(
    private val redis: ReactiveStringRedisTemplate,
    private val objectMapper: ObjectMapper,
) {

    suspend fun append(turn: ChatTurn) {

        val memoryKey =
            "iara:memory:${turn.sessionId}"

        val orderKey =
            "iara:order:${turn.sessionId}"

        val contextKey =
            "iara:context:${turn.sessionId}"

        val json =
            objectMapper.writeValueAsString(turn)

        redis.opsForHash<String, String>()
            .put(
                memoryKey,
                turn.chatHistoryId,
                json,
            )
            .awaitSingle()

        redis.opsForZSet()
            .add(
                orderKey,
                turn.chatHistoryId,
                turn.createdAt.toEpochMilli().toDouble(),
            )
            .awaitSingle()

        redis.opsForHash<String, String>()
            .put(
                "iara:pending",
                turn.chatHistoryId,
                turn.sessionId,
            )
            .awaitSingle()

        redis.delete(contextKey)
            .awaitSingle()
    }
}
```

Assim o service enxerga apenas:

```
append(turn)
```



e não:

```
HSET  
ZADD  
HSET  
DEL
```



Isso mantém a responsabilidade bem separada.

12. O SessionIndexRepository

Depois:

```
</> Kotlin
```



```
request.namUser?.let { user ->  
  
    sessionIndexRepository.touch(  
        userId = user,  
        sessionId = sessionId,  
    )  
}
```

implementação:

```
</> Kotlin
```



```
@Repository
class SessionIndexRepository(
    private val redis: ReactiveStringRedisTemplate,
) {

    suspend fun touch(
        userId: String,
        sessionId: String,
    ) {

        redis.opsForZSet()
            .add(
                "iara:sessions:$userId",
                sessionId,
                Instant.now()
                    .toEpochMilli()
                    .toDouble(),
            )
            .awaitSingle()
    }
}
```

Isso corresponde ao:

```
iara:sessions:{userId}
```



que vimos na Spec.

13. Então o Redis ficará assim

Depois da primeira pergunta:

```
iara:memory:S1
```



```
H1
↓
{
    sessionId: S1,
    chatHistoryId: H1,
    question: "...",
    answer: "...",
    persistStatus: PENDING
}
```

Além disso:

iara:order:S1



score	member
1723940000000	H1

E:

iara:sessions:bruno



score	member
1723940000000	S1

E:

iara:pending



H1 → S1

Enquanto:

iara:context:S1



é invalidado.

14. O próximo request fica ainda mais interessante

Agora Bruno pergunta:

</> JSON



```
{
  "sessionId": "S1",
  "question": "E quais foram os alertas?",
  "namUser": "bruno",
  "context": {
    "customerId": "123456"
  }
}
```

Dessa vez:

```
</> Kotlin

request.sessionId
```



já possui:

```
S1
```



portanto:

```
</> Kotlin

val sessionId =
    request.sessionId ?: UUID.randomUUID().toString()
```



não cria uma nova sessão.

Temos:

```
S1
|
├── H1
|
└── H2 ← novo turno
```



E:

```
iara:memory:S1

H1 → primeiro turno
H2 → segundo turno
```



iara:order:S1 :

```
1000 → H1
2000 → H2
```



E pending :

```
iara:pending

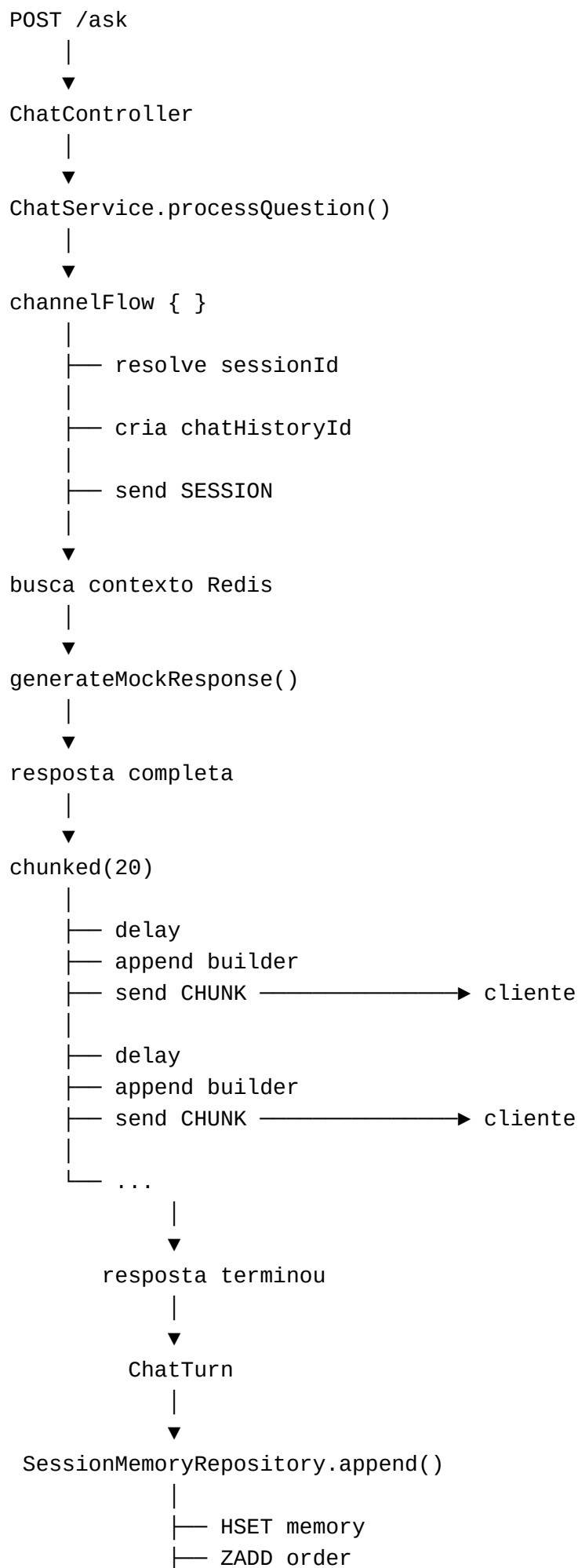
H1 → S1
H2 → S1
```

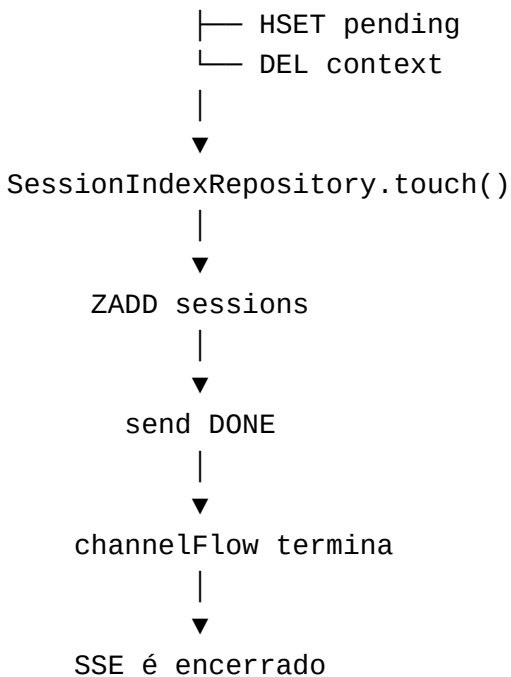


até o reconciliador persistir ambos no Oracle.

15. O fluxo completo dessa implementação

Agora dá para enxergar exatamente a execução:





Esse é o modelo mental que eu usaria para implementar a primeira versão.

E tem uma característica do `channelFlow` que vale guardar: **o código depois do último chunk continua fazendo parte da execução daquela requisição**. Portanto, receber o último texto no frontend não significa necessariamente que o backend terminou. No nosso desenho, ele ainda salva Redis, atualiza a sessão, manda o `done` e somente então o `Flow` é concluído.

Depois disso, em outro fluxo completamente independente, entra o reconciliador:

```

iara:pending
  ↓
H1 → S1
  ↓
iara:memory:S1
  ↓
HGET H1
  ↓
ChatTurn
  ↓
CHAT_SESSION
CHAT_HISTORY
CHAT_TOOL
  ↓
COMMIT
  ↓
HDEL iara:pending H1
  
```



Esse é o ponto em que as peças que você mostrou nas Specs começam efetivamente a se conectar no código.



ChatGPT can make mistakes. Check important info.